

API Automation Interview Practice (Pytest Only)

Question 1 – Validate Paginated Users API

Scenario: GET /users?page={page_number} The API returns: { "page": 1, "total_pages": 5, "data": [...] } Task: Write pytest automation that validates: - status code - response time threshold - page number correctness - schema of each user - no duplicate user IDs across pages 1-3 Guided Thinking: 1. What fixtures would you create? 2. How would you store collected IDs? 3. How would you make the test reusable for different pages? 4. How would you separate validation logic from test logic?

Question 2 – Create User and Verify Persistence

Scenario: POST /users GET /users/{id} Task: Create a user and verify it was persisted correctly.
Validation Requirements: - POST returns expected status code. - Response contains generated ID.
- GET returns the same data. - Email format is valid. - Cleanup after execution if DELETE endpoint exists. Guided Thinking: 1. Which endpoint should be called first? 2. What data should be dynamically generated? 3. How would you avoid hardcoded values? 4. How would you implement cleanup?

Question 3 – Token Expiration Workflow

Scenario: POST /login GET /profile Tokens expire after a short period. Task: Design pytest automation that verifies: - valid token works - expired token fails - correct error message is returned - unauthorized access is blocked Guided Thinking: 1. How would you structure authentication fixtures? 2. How can you simulate expiration? 3. What negative validations are important? 4. Which assertions belong in helper functions?

Question 4 – Data Driven API Contract Testing

Scenario: GET /products/{id} Task: Using pytest parametrization, validate multiple product IDs.

Requirements: - status code validation - mandatory fields validation - type validation - price must be positive - invalid IDs should return expected error response Guided Thinking: 1. Which test cases belong in parametrization? 2. How would you validate response schema? 3. What edge cases would you include? 4. How would you keep the test maintainable when product fields grow?

Question 5 – End-to-End Order Workflow

Scenario: POST /cart POST /checkout GET /orders/{id} Task: Validate the complete order lifecycle.
Requirements: - create cart - add items - checkout successfully - verify order creation - verify totals match expected values Guided Thinking: 1. Which data should be shared between requests? 2. How would you design fixtures for workflow execution? 3. What should happen if checkout fails? 4. Which validations belong at each step?